

```
DROP TABLE IF EXISTS
bu_frd.model_behavior_features_selected_2022_2025;
```

```
CREATE TABLE
bu_frd.model_behavior_features_selected_2022_2025
USING PARQUET
AS
```

```
SELECT
```

```
*,
```

```
/* =====
ACTIVE ACCOUNTS Z
TRAIN P0.1 / P99.9
===== */
```

```
CASE
  WHEN active_accounts_z_3m IS NULL
  THEN NULL
  ELSE GREATEST(
    -13.27906,
    LEAST(
      15.01716,
      active_accounts_z_3m
    )
  )
END AS active_accounts_z_win,
```

```
/* =====
AVG UTILIZATION Z
TRAIN P0.1 / P99.9
===== */
```

```
CASE
  WHEN avg_utilization_z_3m IS NULL
  THEN NULL
  ELSE GREATEST(
    -29.78358,
    LEAST(
      33.76045,
      avg_utilization_z_3m
    )
  )
END AS avg_utilization_z_win,
```

```
/* =====
```

WEIGHTED UTILIZATION Z

TRAIN P0.1 / P99.9

===== */

CASE

WHEN weighted_utilization_z_3m IS NULL

THEN NULL

ELSE GREATEST(

-37.89802,

LEAST(

40.84790,

weighted_utilization_z_3m

)

)

END AS weighted_utilization_z_win

FROM

bu_frd.model_behavior_features_z_2022_2025;

Los thresholds **no se recalculan para 2024 o 2025.**

2. QA final del winsorization

Ejecutaría:

SELECT

COUNT(*) AS rows,

COUNT(DISTINCT party_id) AS parties,

MIN(active_accounts_z_win) AS active_z_min,

MAX(active_accounts_z_win) AS active_z_max,

MIN(avg_utilization_z_win) AS avg_util_z_min,

MAX(avg_utilization_z_win) AS avg_util_z_max,

MIN(weighted_utilization_z_win) AS weighted_util_z_min,

MAX(weighted_utilization_z_win) AS weighted_util_z_max

FROM

bu_frd.model_behavior_features_selected_2022_2025;

Esperamos:

rows = 688,589

parties = 688,589

active:

min >= -13.27906

max <= 15.01716

avg utilization:

min >= -29.78358

max <= 33.76045

weighted:

min >= -37.89802

max <= 40.84790

Con esto terminamos feature engineering.

Paso 24 — ahora sí medimos señal predictiva

WITH long_features AS (

SELECT

party_id,

label_3m,

STACK(

9,

'deterioration_acceleration',

CAST(deterioration_acceleration AS DOUBLE),

'adverse_acceleration',

CAST(adverse_acceleration AS DOUBLE),

'active_accounts_change',

CAST(active_accounts_change_vs_baseline AS DOUBLE),

'balance_change',

CAST(balance_change_vs_baseline AS DOUBLE),

'past_due_change',

```

        CAST(past_due_change_vs_baseline AS DOUBLE),

        'delinquency_share_change',
        CAST(delinquency_share_change_vs_baseline AS DOUBLE),

        'avg_utilization_change',
        CAST(utilization_change_vs_baseline AS DOUBLE),

        'weighted_utilization_change',
        CAST(weighted_utilization_change_vs_baseline AS DOUBLE),

        'max_dpd_jump_recent',
        CAST(max_dpd_jump_recent_3m AS DOUBLE)

    ) AS (feature_name, feature_value)

FROM
    bu_frd.model_behavior_features_selected_2022_2025

WHERE model_split = 'TRAIN'
)

SELECT
    feature_name,
    label_3m,

    COUNT(*) AS parties,

    SUM(
        CASE WHEN feature_value IS NOT NULL
            THEN 1 ELSE 0 END
    ) AS available,

    ROUND(
        100.0 *
        SUM(
            CASE WHEN feature_value IS NOT NULL
                THEN 1 ELSE 0 END
        ) / COUNT(*),
        2
    ) AS available_pct,

    AVG(feature_value) AS mean_value,

    percentile_approx(
        feature_value,
        0.50
    ) AS median_value,

```

```

percentile_approx(
  feature_value,
  0.25
) AS p25,

percentile_approx(
  feature_value,
  0.75
) AS p75

FROM long_features

GROUP BY
  feature_name,
  label_3m

ORDER BY
  feature_name,
  label_3m DESC;

```

5. Para Z-score hacemos el mismo diagnóstico

```

WITH long_z AS (

  SELECT
    party_id,
    label_3m,

    STACK(

      3,

      'active_accounts_z',
      CAST(active_accounts_z_win AS DOUBLE),

      'avg_utilization_z',
      CAST(avg_utilization_z_win AS DOUBLE),

      'weighted_utilization_z',
      CAST(weighted_utilization_z_win AS DOUBLE)

    ) AS (feature_name, feature_value)

```

```

FROM
    bu_frd.model_behavior_features_selected_2022_2025

WHERE model_split = 'TRAIN'
)

SELECT
    feature_name,
    label_3m,

    COUNT(*) AS parties,

    SUM(
        CASE WHEN feature_value IS NOT NULL
            THEN 1 ELSE 0 END
    ) AS available,

    ROUND(
        100.0 *
        SUM(
            CASE WHEN feature_value IS NOT NULL
                THEN 1 ELSE 0 END
        ) / COUNT(*),
        2
    ) AS available_pct,

    AVG(feature_value) AS mean_value,

    percentile_approx(
        feature_value,
        0.25
    ) AS p25,

    percentile_approx(
        feature_value,
        0.50
    ) AS median_value,

    percentile_approx(
        feature_value,
        0.75
    ) AS p75

FROM long_z

GROUP BY
    feature_name,

```

label_3m

ORDER BY

feature_name,
label_3m DESC;